Klausur SS12

1. **Zahlensysteme**

   1. **Stellen Sie die Hexadezimale Zahl 0x3A im Duodezimal (Basis 12) System dar. (Rechenweg!)**

      $3*16^1+10*16^0=48+10=58$

      $$\frac{58}{12} = 4 \quad Rest \quad 10$$
      $$\frac{4}{12} = 0 \quad Rest \quad 4$$

      Ergebnis: 4A

   2.

      1. **Wieviele Stellen benötigt man um die Hexadezimale Zahl 0xFFA2 Binär darsustellen? (Herleitung!)**

         Bei Hexadezimalen Zahlen bedeutet jede Ziffer 4 Bit im Binärsystem.
         4 Ziffern ergeben dann folglich 16 Binäre Stellen.

      2. **Was ist die größte Zahl, die man in einer ganzzahligen, vorzeichenlosen 64 Bit Variable speichern kann?**

         $2^{64}-1=1{,}84467441*10^{19}-1$

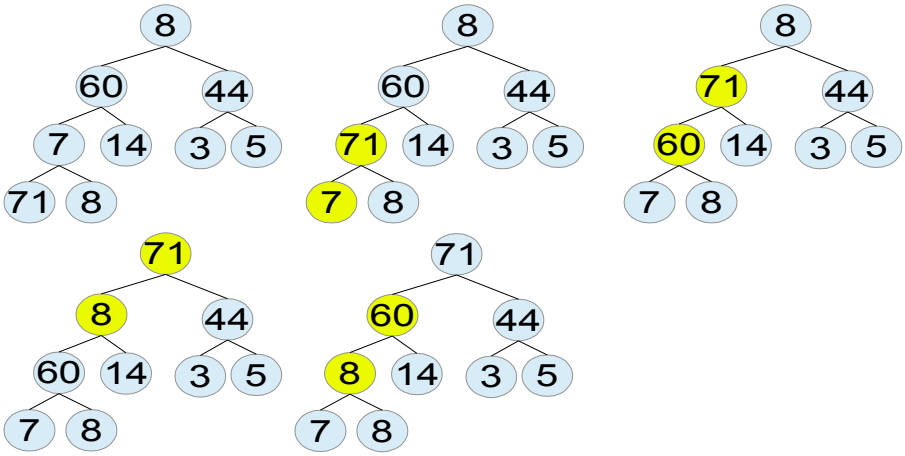
      3. **Um wie viel ist eine 66 Bit Zahl größer als eine 64 Bit Zahl (beide vorzeichenlos und ganzzahlig)?**

         Um das 4-Fache.

2. **Sortieralgorithmen**

   1. **Nutzen Sie die Build-Heap Funktion aus dem Skript um aus diesem Array einen gültigen Heap zu erzeugen. Stellen Sie den Ablauf grafisch dar und markieren Sie jeweils das von Build Heap aktuell behandelte Element.**

8	**60**	**44**	**7**	**14**	**3**	**5**	**71**	**8**

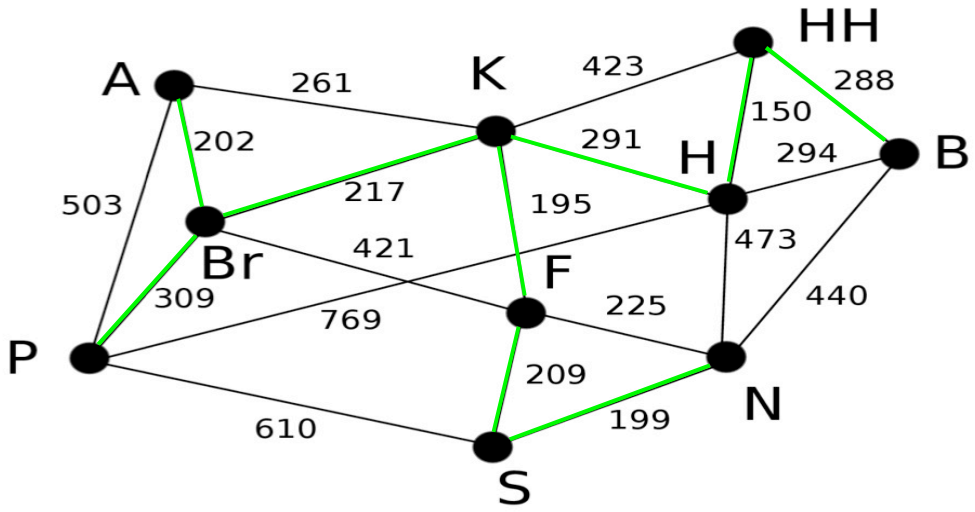


2. **Warum wird das Pivot-Element bei Quicksort idealerweise zufällig gewählt?**

   Die Wahrscheinlichkeit, dass ein randomisiertes Quicksort den Worst-Case trifft ist sehr klein. Aber die zufällige Permutation der Liste vor dem Sortieren kostet Rechenzeit. Daher: Zufällige Wahr des Pivotelements.

3. **Algorithmen auf Graphen**

   Gegeben ist folgender ungerichteter Graph G_1



1. **Geben Sie die Knotenmenge V an (Formal korrekte Darstellung!)**

   V = {A, K, HH, H, B, Br, F, P, S, N}

2. **Zeichnen Sie den Minimal Aufspannenden Baum (MST) on den oben angegebenen Graphen** G_1 **ein und geben Sie den MST als Menge** E_{MST} **an.**

   E_{MST} = {(S, N), (S, F), (F, K), (K, H), (H, HH), (HH, B), (K, Br), (Br, A), (Br, P)}

3. **Bestimmen Sie eine (Näherungs-)Lösung des TSP Problems auf den Graph** G_1 **mit Hilfe des Nearest-Neighbour-Algorithmus mit Startknoten A. (Notation U → R → K → U)**

   A → Br → K → F → S → N → B → HH → H → P → A

4. Hashing

Gegeben ist die Hashfunktion h(x) = x mod 9.

Schlüssel	Hash-Wert
19	1
$x*9+4$	**4**
180	0
48751	7
27	0
$x*9+7$	**7**

1. Füllen Sie die Tabelle aus.

Nicht „fett“ geschriebenes ist von mir eingetragen. „Fett“ geschriebenes ist gegeben.

2. Beschreiben Sie mindestens zwei Strategien um mit Kollisionen umzugehen.

Kollisionsauflösung durch Verkettung:

Dabei wird bei bei einer Kollision in einem Feld eine weiteres Feld an dieses angehängt (Array von Feldern) und für die Speicherung des Elements verwendet.

Offene Adressierung:

Hier wird bei einer Kollision über einen bestimmten Algorithmus solange ein neuer Hashwert ermitteln, bis ein freies Feld gefunden wurde in dem dann das Element gespeichert wird.

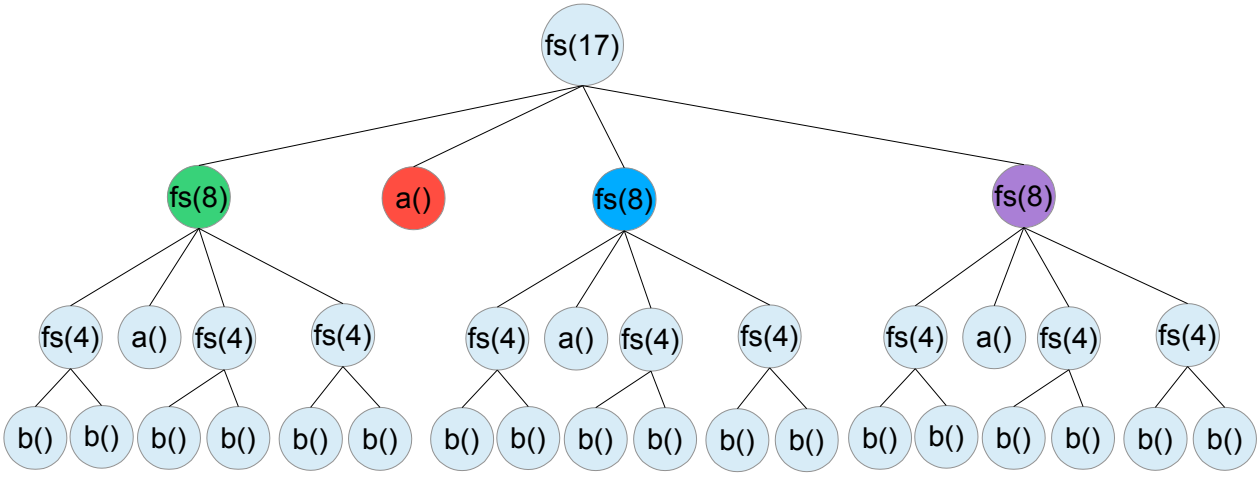
5. Algorithmische Komplexität

Gegeben sei der folgende Ausschnitt eines C Programms:

```
void fs(int x) {

    if(x > 4) {
        fs(x/2);
        printf("a");
        a();
        fs(x/2);
        fs(x/2);
    } else {
        printf("b");
        b();
        b();
    }
}
```

1. **Zeichnen Sie den Rekursionsbaum der Funktion fs(17) inklusive den nicht rekursiven Funktionen a und b.**



2. **Welche Ausgabe erzeugt die Funktion fs(19)?**

   babbababbbabb

3. **Berechnen Sie die Komplexität des Algorithmus mit Hilfe des Master-Theorems. Die Funktionen a und b besitzen beide n^2 Komplexität.**

$$a=3, \quad b=2, \quad f(n)=n^2$$
$$1: \quad n^{\log_b a} = n^{\log_2 3} = n^{1,58}$$
$$2: \quad n^{1,58} < n^2$$
$$\text{Mastertheorem Fall 3:} \quad T(n)=\Theta(f(n))=\Theta(n^2)$$

6. Asymptotik, Komplexität

1. **Kreuzen Sie in der folgeneden Tabelle alle zutreffenden Felder an, so dass die Landau'schen Symbole eine Verdeutlichung für die Schranke g(n) zu f(n) sind. Beispiel: f(n) = ω(g(n))**

f(n)	g(n)	O	o	Ω	ω	Θ	KA*
$n*\sin(n)$	n						X
$\log(n^2)+r^q$	n	X	X				
$n!*n!$	n^n			X	X		
$n^2+\tan(n)$	$n^{1/4}$			X	X		
$(n^m)^n$	n^{nm}	X		X		X	

***KA = Keine Aussage möglich**

7. Sprachen und Automaten

Die Sprache L_1 ist definiert durch fogenden regulären Ausdruck:

a (b {2, 2} c? | bac+)

1. **Erstellen Sie eine Grammatik G_1 die die Sprache L_1 erzeugt. (Formal Korrekt!)**

   G_1 = ({S, A, B, C, D}, {a, b, c}, P, S)

   P = {S → aA, A → bB, B → bC | b, C → c,

   B → aD, D → cD | c}

2. **Erzeugen Sie einen deterministischen Automaten M_1 der die Sprache L_1 akzeptiert. (Wählen Sie eine Beschreibungsform aus dem Skript)**

   M_1 = ({A, B, C, D, E, F, e}, {a, b, c}, δ, A, {D, e, F})

δ	A	B	C	D	E	F	e
a	B		E				
b		C	D				
c				e	F	F	

3. **Enthält die Sprache L_1 Wörter der Länge 2? Wenn ja, welche?**

   Nein, das kürzeste Wort ist abb.

4. **Enthält die Sprache L_1 Wörter der Länge 4? Wenn ja, welche?**

   Ja: abbc, abac

5. **Welche Länge besitzt das längste Wort der Sprache L_1?**

   Eine unendliche Länge, denn das c+ besagt, dass beliebig viele c hintereinander kommen dürfen. Z.B.: abacccccccccccccc…c_∞